Manuscript as of August 09, 2025

System 2 through Okazaki-RAG:

Capacity-Bounded Inline Fragment Manipulation for Deliberative Reasoning

Dyuti Ghosh, Dibya Ghosh, Debi Prasad Ghosh **debiprasadghosh@gmail.com**

Abstract: We argue that large language models (LLMs) can realize System 2-style deliberation without requiring larger networks or external agentic frameworks. This can be achieved by pairing a capacity-bounded controller with representation-level retrieval and assembly. Building on the Okazaki-RAG blueprint, we formalize a dual-strand architecture where a lightweight Context Strand maintains a global "gist" of the evolving problem, emitting a SoftMax distribution over a library of *Okazaki fragments*, pre-computed last-layer KV-cache blocks. Guided by this signal, a high-fidelity Value Strand performs reasoning over a small, actively curated working set of these fragments. We ground this design in classic working-memory findings, treating the working set size C as a hard architectural budget (Miller, 1956; Cowan, 2001), which forces the selective, sequential operations that mirror human System 2 cognition.

We introduce an Inline Manipulation Engine (IME) that performs reversible, primitive operations, {Append, Remove, Assemble, Swap}, on the working set. The IME evaluates candidate assemblies via a beam-bounded search scored by a look-ahead likelihood from the Value Strand. Crucially, the Assemble operation manipulates representations, not text. Fragments are "ligated" using position-independent cache reuse (e.g., EPIC/PIC), with quality restored via selective recomputation (e.g., CacheBlend) and latency minimized by offline precomputation (e.g., TurboRAG). This technical stack makes iterative, fragment-level search practical at interactive speeds. Conceptually, our controller internalizes popular deliberation protocols like Chain-of-Thought and Tree-of-Thoughts. Instead of emitting textual "thoughts," the model edits and evaluates assemblies of fragments directly in latent space. We position this approach relative to explicit-memory and long-context systems, clarifying how composing pre-digested computational blocks under a tight capacity budget differs from concatenating text or gating past states. As a conceptual contribution, we define the controller formalism, analyze its complexity under modern KV-reuse paradigms, and surface key risks, including approximation errors from KV injection and the challenges of provenance tracking.

1. Introduction

Large language models (LLMs) have become formidable System 1 engines—fast, fluent pattern completers—yet they falter on tasks demanding deliberate search, compositional assembly of knowledge, and robust working-memory control. This gap is framed by dual-process theory, which distinguishes intuitive, automatic System 1 cognition from the slow, effortful, rule-guided reasoning of System 2 (Kahneman, 2011). A plausible System 2 substrate must also adhere to classic cognitive constraints: the capacity of the human focus of attention is surprisingly small, historically estimated at "seven, plus or minus two" items (Miller, 1956) and, under more controlled conditions, closer to four "chunks" of information (Cowan, 2001).

From an engineering standpoint, simply enlarging context windows has not guaranteed reliable long-range reasoning; models often under-utilize information "lost in the middle" of long inputs (Liu et al., 2023). Meanwhile, Retrieval-Augmented Generation (RAG) grounds outputs in external knowledge but

often relies on the shallow concatenation of retrieved text, incurring significant prefill latency (Gao et al., 2023).

A path forward emerges from a convergence of cognitive architecture and serving-stack innovation. Recent breakthroughs in LLM serving—such as position-independent caching (Hu et al., 2024), selective recomputation (Yao et al., 2024), and precomputed per-chunk KV retrieval (Lu et al., 2024)—demonstrate that *representations*, not raw text, can be the fundamental unit of retrieval and reuse. This paper sharpens the cognitive and systems perspective of Okazaki-RAG, a dual-strand, parameter-efficient architecture that leverages these advances (Ghosh et al., 2025). Okazaki-RAG separates a lightweight **Context Strand** (for global gist) from a high-fidelity **Value Strand** (for reasoning on precomputed KV-cache "Okazaki fragments"). By proposing an explicit controller that selects and assembles a capacity-limited working set of these fragments, we can transform the passive context window into an active, deliberative workspace.

1.1 Literature survey

Cognitive foundations

System-1/2 dual-process theory (Kahneman, 2011) provides a compelling framework for architecting LLMs, a direction explored by recent work on neuro-symbolic models and agentic frameworks (Gronchi & Perini, 2024; Zhang et al., 2025). Efforts to align LLMs with this cognitive duality aim to incentivize more efficient and deliberate reasoning (Cheng et al., 2025; Ziabari et al., 2025). Foundational to this is the concept of a capacity-limited workspace. Classic results establish this limit at "7±2" items (Miller, 1956) or, with rehearsal controlled, a "magical number four" (Cowan, 2001; Cowan, 2015), suggesting that any plausible System-2 controller must operate over a small, actively managed slate.

Long-context limits

While architectural advances have expanded context windows, mere length does not guarantee effective use. A robust body of work shows that LLMs exhibit strong positional biases, often under-utilizing information presented in the middle of long prompts—a phenomenon termed "lost in the middle" (Liu et al., 2023). Subsequent studies have confirmed this bias and explored methods for mitigation, such as calibrating attention or employing decompositional training, underscoring the need for explicit selection and reassembly rather than passive concatenation (Hsieh et al., 2024; He et al., 2023; Yu et al., 2024; Leng et al., 2024).

RAG and explicit memory.

Retrieval-Augmented Generation (RAG) (Lewis et al., 2020) has become a standard paradigm, with numerous surveys mapping its evolution, techniques, and future directions (Gao et al., 2023; Huang & Huang, 2024; Gupta et al., 2024; Zhao et al., 2024). Architectures with explicit memory have demonstrated significant parameter efficiency. **RETRO** conditions generation on retrieved text via chunked cross-attention (Borgeaud et al., 2021), a concept extended by works like InstructRetro (Wang et al., 2023). **Memorizing Transformers** use a non-differentiable kNN lookup over past (key, value) states at inference (Wu et al., 2022). **Transformer-XL** extends context by reusing cached states from previous segments in a recurrent manner (Dai et al., 2019). Together, these lines point to addressable memory as a core primitive but generally stop short of a controlled, capacity-bounded manipulation regime.

Reasoning protocols

Prompt-level techniques elicit step-by-step or branching deliberation but expend tokens and keep reasoning externalized in text. Foundational methods include **Chain-of-Thought** (CoT) (Wei et al., 2022), which generates intermediate reasoning steps, and **Tree-of-Thoughts** (ToT) (Yao et al., 2023), which explores multiple reasoning paths. These have been extended with techniques like **Self-Consistency** (Wang et al., 2022), which samples diverse paths and takes a majority vote, and **Least-to-Most** prompting (Zhou et al., 2022), which decomposes problems into simpler sub-steps. Our work aims to internalize these deliberative patterns at the representation level. Modular RAG (Gao et al., 2024).

Serving breakthroughs

The feasibility of our proposed inline manipulation rests on recent serving-time advances that make representation-level reuse practical. **TurboRAG** precomputes per-document KV caches offline to nearly eliminate online prefill latency (Lu et al., 2024). **EPIC** enables position-independent caching (PIC), allowing cached KV segments to be reused regardless of their original position (Hu et al., 2024). **CacheBlend** addresses the resulting quality degradation by selectively recomputing a small subset of tokens to efficiently restore cross-attention across independently cached chunks (Yao et al., 2024). This stack transforms KV fragments into first-class building blocks for iterative reasoning.

Okazaki-RAG

The prior paper introduces the core dual-strand design, the concept of Okazaki fragments (final-layer KV blocks with summary keys), and the late-fusion mechanism that allows a lightweight controller to steer a high-fidelity reasoning strand (Ghosh et al., 2025). This work builds directly on that substrate to formalize the System-2 control loop.

1.2 Paper contribution

This conceptual companion paper advances "System 2 through Okazaki-RAG" along four axes:

1. **Cognitive-constrained control.** We formalize a capacity budget $C \in [4,9]$ for the working set W_t, aligning with human working-memory bounds and forcing selective, serial manipulation—an essential ingredient of System-2 style deliberation (Miller, 1956; Cowan, 2001).
2. **Inline Manipulation Engine (IME).** We specify a representation-level controller that performs {Append, Remove, Assemble, Swap} on Okazaki fragments guided by a SoftMax over fragment summaries from the Context Strand, with beam-bounded search and value-guided evaluation. This internalizes CoT/ToT-like search without emitting textual thoughts (Wei et al., 2022; Yao et al., 2023).
3. **Representation-augmented generation.** We argue for treating KV fragments as the unit of retrieval and composition (cf. TurboRAG, CacheBlend, EPIC), and we map how PIC + selective recompute make position-independent assembly both feasible and accurate enough for iterative search (Lu et al., 2024; Yao et al., 2024; Hu et al., 2024).
4. **Scope and desiderata.** We position Okazaki-RAG relative to RETRO, Memorizing Transformers, and Transformer-XL, clarifying the locus of deliberation (external prompting vs. internal manipulation) and outlining evaluation and safety desiderata (provenance tracking, search-budget scheduling, and secure KV stores).

1.3 Paper structure

Section 2 recaps the Okazaki-RAG dual-strand substrate and defines Okazaki fragments as actionable units of knowledge. Section 3 introduces the Inline Manipulation Engine and its search policy. Section 4 situates the framework among RAG, RETRO, Memorizing Transformers, and Transformer-XL. Section 5 connects the design to serving primitives (EPIC, CacheBlend, TurboRAG). Section 6 aligns IME operations with CoT/ToT-style protocols. Section 7 critically examines latency/quality trade-offs, fidelity risks of mid-sequence KV injection, provenance, and security. Section 8 summarizes contributions and outlines research trajectories for controller training and adaptive deliberation.

Prior art basis. This paper explicitly builds on Okazaki-RAG: A Dual-Strand, Parameter-Efficient Architecture for Long-Context Modeling (Ghosh et al., 2025).

2. The Okazaki-RAG Substrate

Okazaki-RAG is a dual-strand architecture that decouples low-cost global context maintenance from selective, high-fidelity reasoning, fusing them only at the final prediction layer.

2.1 Dual-Strand Architecture

- **Context Strand:** A small, lightweight decoder that continuously attends to the full conversation history. Its function is not detailed recall but *control*: it computes a compressed "gist" of the discourse state and generates ranking signals over the fragment library. Its small width ensures its KV-cache footprint scales gently, allowing it to scan long histories without the prohibitive prefill cost of monolithic LLMs.
- **Value Strand:** A frozen, high-capacity LLM used as a quality feature extractor and generator. It operates discontinuously on a small, assembled set of fragments rather than raw text. By retrieving pre-computed KV blocks, it avoids online encoding and focuses its computational power on reasoning over the curated working set.

Representations from both strands are fused late in the process, enabling parameter-efficient training focused on the Context Strand and the fusion layer (Ghosh et al., 2025).

2.2 Okazaki Fragments: Actionable Units of Knowledge

The fundamental unit of knowledge in this framework is the **Okazaki fragment**, an actionable representation produced offline from a source text span:

$$\text{frag}_i = (K_i^{(L)}, V_i^{(L)}, \mathbf{e}_i, \text{meta}_i)$$

Here, $K_i^{(L)}$ and $V_i^{(L)}$ are the final-layer Key-Value tensors from a frozen LLM; e_i is a compact embedding for efficient similarity search; and meta_i stores provenance data (e.g., document ID, offsets). The name is an analogy to lagging-strand DNA replication, where short, independently synthesized fragments are later ligated into a continuous strand by a coordinated enzymatic process. Similarly, our architecture assembles pre-computed knowledge fragments into a coherent reasoning context.

This representation-first retrieval avoids online prefill costs, a paradigm shared with systems like TurboRAG (Lu et al., 2024). At inference, the controller ranks fragments via a query q_t against the embeddings e_i and admits a capacity-bounded set into the working slate for assembly.

3. The Inline Manipulation Engine (IME)

The IME is the control core that executes capacity-bounded, step-wise deliberation. It treats Okazaki fragments as actionable objects, editing a small working set within the active context and scoring alternatives with look-ahead passes through the Value Strand.

3.1 State, Objects, and Invariants

- **Fragments:** As defined in §2.2, fragments are tuples of (KV-cache, summary embedding, metadata).
- **Controller Signal:** At each step *t*, the Context Strand emits a query q_t. Relevance scores $u_i=\langle q_t, e_i\rangle$ are computed, and a SoftMax distribution $p_i=\text{softmax}(u)_i$ is formed over all fragments. This distribution represents the controller's attentional allocation.
- **Working Set:** $W_t \subset F$ is the slate of fragments under active consideration. Its size is constrained by the capacity budget: $|W_t| \leq C$, where $C \in [4, 9]$ is chosen based on cognitive findings (Miller, 1956; Cowan, 2001).

3.2 Primitive Operations

The IME operates on the working set using a minimal, reversible API:

- **Append(f):** Inserts a high-probability candidate fragment f into W_t if $|W_t| < C$.
- **Remove(f):** Evicts the least useful fragment (e.g., lowest p_i) to free capacity.
- **Assemble(S):** Ligates a subset $S \subseteq W_t$ into an ordered assembly, materializing a position-independent KV view for the Value Strand.
- **Swap(f_i, f_j):** Reorders elements within an assembly to test alternative compositions.

These primitives are the latent analogues of the textual "thought" steps in protocols like Chain-of-Thought (Wei et al., 2022) and Tree-of-Thoughts (Yao et al., 2023), executed directly on representations.

3.3 Search Policy

The IME performs a beam-bounded look-ahead search over possible manipulations:

1. **Candidate Pool:** Form a candidate set C_t from the top-*k* fragments (ranked by p_i) and the current working set W_t.
2. **Move Generation:** Enumerate legal moves over (W_t, C_t) using the primitive operations, respecting the capacity constraint.
3. **Scoring & Pruning:** For each successor state W', compute a score:

$$\text{score}(W') = \alpha \cdot V(W') + (1-\alpha) \cdot \sum_{f \in W'} p_f - \lambda \cdot \Omega(W')$$

   where V(W') is a look-ahead value (e.g., next-token log-likelihood from the Value Strand), the sum over p_f preserves global relevance, and Ω(W') is an optional regularization term. The top-*B* states (where *B* is the beam width) are retained.

4. **Commit / Backtrack:** The best successor becomes the new state W_{t+1}. If its value drops below a threshold, the engine can backtrack to explore another branch, emulating ToT-style search (Yao et al., 2023).

The dominant cost of this search is the look-ahead evaluation V(W'). By leveraging modern serving primitives, this step reduces to a final-layer pass plus a small boundary recomputation, rather than a full prefill of retrieved text. The deliberation budget, controlled by beam width *B* and search depth *D*, becomes the primary trade-off between solution quality and latency.

```
Input: capacity C, beam B, k, weights α, λ
State: W = ∅
repeat
  q = ContextStrand.query()
  p = softmax(sim(q, s_i) for s_i in index)      # controller distribution
  Cands = top_k(p, k) ∪ W
  Beam = {W}
  for depth = 1..D:
    Next = {}
    for S in Beam:
      for op in {Append, Remove, Assemble, Swap} over (S, Cands) respecting |S| ≤ C:
        S' = op(S)
        V = ValueStrand.lookahead(S')             # run on assembled KV
        score = α·V + (1-α)·Σ_{f∈S'} p[f] - λ·Ω(S')
        Next.add(S', score)
    Beam = top_B(Next)
  W = argmax Beam
  emit_next_token(ContextStrand, ValueStrand, W)     # late fusion decode
until end
```

4. Positioning Among Memory-Augmented Architectures

Okazaki-RAG's design choices distinguish it from other prominent memory and long-context paradigms.

- **Versus Standard RAG:** Classic RAG retrieves text and re-encodes it online, incurring high prefill costs and suffering from "lost-in-the-middle" effects in long prompts (Lewis et al., 2020; Liu et al., 2023). Okazaki-RAG retrieves *representations* and manipulates them directly, turning retrieval results into actionable compute.

- **Versus RETRO:** The Retrieval-Enhanced Transformer (RETRO) retrieves text and *conditions* the generator via a separate cross-attention pathway (Borgeaud et al., 2021). In Okazaki-RAG, the retrieved fragments are not just side-information; they are *ligated* to form the primary context over which the Value Strand reasons.

- **Versus Memorizing Transformers:** Memorizing Transformers use k-Nearest Neighbor lookup over past internal (K, V) states and combine them via simple gated sums (Wu et al., 2022). Okazaki-RAG replaces this with a more structured search over externally indexed fragments, governed by the IME's explicit compositional operations.

- **Versus Transformer-XL:** Transformer-XL uses segment-level recurrence to pass a monolithic state from the immediately preceding segment, creating a linear memory (Dai et al., 2019).

Okazaki-RAG's controller is non-linear, selecting and assembling fragments from anywhere in the knowledge base based on semantic relevance.

Axis	Standard RAG	RETRO	Memorizing Transformer	Transformer-XL	Okazaki-RAG
Unit Retrieved	Text chunks	Text chunks	Past (K,V) pairs	Previous segment states	Final-layer KV fragments
Integration	Concatenate in prompt	Chunked cross-attention	kNN-gated mixing	State reuse (recurrence)	Position-independent assembly
Controller	External retriever	External retriever	kNN lookup	Fixed recurrence	Context-Strand SoftMax → IME
Deliberation	External (prompting)	External (prompting)	None (gated mixing)	None (linear state)	Internal (IME search)
Latency Tactics	None (full prefill)	—	—	State reuse	Offline KV, PIC, selective repair

5. Serving primitives that make Okazaki-RAG practical (EPIC, CacheBlend, TurboRAG)

Okazaki-RAG's Inline Manipulation Engine (IME) assumes that the system can (i) fetch representation-level units (Okazaki fragments), (ii) assemble them even when they are not input prefixes, and (iii) repair any accuracy loss from such reuse—fast enough to sit inside a search loop. That capability is not hypothetical; it is exactly what three recent serving primitives deliver:

- **TurboRAG:** precompute and store per-chunk KV caches offline; at inference, retrieve KVs directly to all but eliminate online prefill. (Lu et al., 2024).
- **EPIC / PIC:** position-independent caching so cached KVs can be reused even when chunks are not prefixes; includes algorithms (e.g., AttnLink, LegoLink) to minimize recomputation. (Hu et al., 2024).
- **CacheBlend:** selective recomputation of a small subset of tokens to restore cross-attention when independently cached chunks are linked. (Yao et al., 2024). Together, they transform fragments from retrieved text into actionable compute blocks that IME can rank, assemble, and evaluate at interactive speeds.

5.1 Position-independent assembly with EPIC (PIC)

Problem. Classic cache reuse needs an exact prefix match; RAG/long-context workloads rarely have one.

EPIC formalizes position-independent context caching (PIC) so a cached chunk's KV can be placed anywhere in the sequence, then "linked" with AttnLink/LegoLink to recover accuracy with minimal

recomputation. Reported wins include up to 8× lower TTFT and 7× higher throughput versus prefix-only baselines, with negligible accuracy loss; LegoLink further cuts TTFT (up to ≈3× vs. CacheBlend in their analysis) while bounding accuracy deltas. (Hu et al., 2024).

Role in Okazaki-RAG. IME's Assemble primitive treats EPIC as the linker: it creates a position-independent KV view over a chosen slate of fragments so the Value Strand can attend them as if they were computed together. This is what makes discontinuous, order-sensitive composition feasible inside the context window, not just as side conditioning.

5.2 Accuracy repair with CacheBlend (selective KV recompute)

Problem. Linking independently computed KVs misses cross-attention across chunk boundaries; naïvely concatenating cached KVs degrades quality.

CacheBlend. Reuse cached KVs regardless of position, then recompute only a small subset of tokens (boundary and high-impact positions) to partially update each cache. Across multiple LLMs/benchmarks, CacheBlend reports 2.2–3.3× faster TTFT and 2.8–5× higher throughput vs. full prefill, while matching generation quality—and can pipeline recompute with KV retrieval. (Yao et al., 2024).

Role in Okazaki-RAG. IME marks repair sets whenever a candidate assembly is formed; the Value Strand's look-ahead call can request CacheBlend-style repairs only on the current assembly's boundaries, preserving the search loop's responsiveness.

5.3 KV-first retrieval with TurboRAG (offline precompute)

Problem. Standard RAG pays the full prefill cost for every retrieved chunk, inflating TTFT.

TurboRAG. Move text→KV encoding offline: precompute and index per-chunk KV caches, then retrieve KVs at inference. With appropriate masking/position handling (e.g., RoPE-compatible schemes) and a light fine-tune, TurboRAG reports up to 9.4× TTFT reduction (≈8.6× on average) while retaining accuracy. (Lu et al., 2024).

Role in Okazaki-RAG. Okazaki fragments are TurboRAG-style KVs augmented with a summary embedding and provenance. IME queries summaries to propose a capacity-bounded slate; the Value Strand consumes the corresponding KVs immediately—no online prefill.

5.4 A unified serving blueprint for Okazaki-RAG

- **Offline indexing (TurboRAG-style).** For each document span, compute and store final-layer KVs + a summary vector + metadata. This is the fragment library. (Lu et al., 2024).
- **Online candidate selection.** The Context Strand emits a query; top-k summaries define the fragment candidate pool.
- **Assembly (EPIC).** IME's Assemble invokes PIC to materialize a position-independent KV view over the current working set (order matters). (Hu et al., 2024).
- **Boundary repair (CacheBlend).** If needed, selectively recompute a small token subset on assembly boundaries before evaluation. (Yao et al., 2024).
- **Look-ahead scoring.** Run a short Value Strand pass on the assembled KV view; combine with the controller prior to score the move; beam-prune.

- **Late fusion & decode.** Fuse Context/Value states for next-token prediction. This stack makes latent search over assemblies fast and faithful enough to live inside decoding rather than as an external agent loop.

5.5 Latency–quality–budget calculus (what to tune)

- **TTFT floor.** With KV-first retrieval (TurboRAG), the TTFT baseline drops by up to ≈9× relative to text-first RAG; this headroom funds IME's look-ahead evaluations. (Lu et al., 2024).
- **Assembly cost.** EPIC amortizes non-prefix reuse; LegoLink/AttnLink reduce recomputation, keeping accuracy near full prefill. (Hu et al., 2024).
- **Repair budget.** CacheBlend exposes a knob: how many tokens to recompute per boundary. Increasing this raises latency but tightens approximation to full prefill. (Yao et al., 2024). In Okazaki-RAG, these translate to (beam, depth, repair-rate)—the same "deliberation budget" prompt-level ToT manipulates, but now in latent space and backed by serving primitives.

5.6 Practical considerations: provenance and position bias

- **Provenance.** Store fragment IDs and source offsets in metadata and propagate them through assemblies so outputs can expose fragment-level citations. (This is essential when composing multiple caches.)
- **Position sensitivity.** Because LLMs under-use middle context ("lost-in-the-middle"), a capacity-bounded slate plus controller scoring prevents drowning crucial evidence in long, flat prompts—one of the motivations for assembly rather than raw concatenation. (Liu et al., 2023).

Takeaway. EPIC (PIC), CacheBlend, and TurboRAG are not peripheral optimizations; together they are the execution substrate that lets Okazaki-RAG treat knowledge as pre-digested compute. They enable the IME to assemble, repair, and score fragment hypotheses inline, turning representation-level retrieval into a practical System-2 control loop.

6. Aligning IME operations with CoT/ToT-style protocols

The Inline Manipulation Engine (IME) is designed to internalize popular prompting protocols—Chain-of-Thought (CoT), Self-Consistency, Least-to-Most (LtM), and Tree-of-Thoughts (ToT)—by replacing textual thoughts with representation-level edits to Okazaki fragments inside the context window. Instead of emitting long token sequences, IME composes, reorders, prunes, and scores assembled KV fragments under a tight working-memory budget, then commits the best move—mirroring the deliberation patterns that CoT/ToT elicit externally. This section makes the mapping explicit and clarifies where Okazaki-RAG's dual-strand substrate provides the necessary control signals and evaluation hooks (Ghosh et al., 2025).

6.1 Conceptual mapping: from textual "thoughts" to latent edits

CoT ⇒ sequential step expansion.

CoT improves complex reasoning by prompting models to generate intermediate steps before the answer (e.g., GSM8K, arithmetic/commonsense gains). In IME, a step is the transition $W_t \rightarrow W_{t+1}$ produced by one primitive—typically Append (admit a high-mass fragment) followed by Assemble

(ligate a subset into the active reasoning context). The "reasoning trace" becomes the ordered sequence of working sets rather than a paragraph of tokens. (Wei et al., 2022).

Self-Consistency ⇒ ensemble over assemblies.

Self-Consistency samples multiple CoT paths and selects a majority/aggregated answer. IME achieves the same effect by keeping a beam of top-B latent assemblies and either (a) marginalizing scores across the beam during decode, or (b) decoding per-beam and voting on the final answer—an internal analogue of diverse reasoning paths. (Wang et al., 2022).

Least-to-Most ⇒ decomposition schedule.

LtM first decomposes a problem into simpler subproblems, solves them in sequence, and composes the results. IME implements this as a curriculum over fragments: early steps restrict candidates to "easy" fragments (high SoftMax mass from the Context Strand), then Append/Assemble promotes those partial results into the slate before harder fragments are considered, enforcing a decomposition-first search order. (Zhou et al., 2022; Ghosh et al., 2025).

ToT ⇒ branching, evaluation, and backtracking.

ToT generalizes CoT by exploring multiple branches, self-evaluating states, and backtracking. IME mirrors this with beam-bounded branching over {Append, Remove, Assemble, Swap}, look-ahead scoring via the Value Strand on the assembled KVs, and rollback when scores deteriorate—i.e., a tree search whose nodes are latent assemblies rather than text strings. (Yao et al., 2023).

6.2 Primitive-level correspondences

Append ↔ "next thought."

Bringing a salient fact into working memory corresponds to writing the next CoT sentence; IME admits a fragment ranked by the Context Strand's SoftMax over summary embeddings. (Ghosh et al., 2025).

Remove ↔ pruning.

ToT often prunes weak branches; IME evicts low-value fragments to respect the capacity C and sharpen subsequent search. (Yao et al., 2023).

Assemble ↔ composition step.

Where CoT composes textual premises, IME ligates KV fragments into a position-independent view that the Value Strand can immediately score, enabling look-ahead without re-encoding text. (Ghosh et al., 2025).

Swap ↔ reordering hypotheses.

Reordering premises can flip entailment or calculation order; IME tests alternative orders of the same fragments with negligible additional prefill cost.

6.3 State evaluation: from textual self-checks to latent look-ahead

CoT/ToT rely on self-evaluation—either explicit rubric prompts or heuristic checks—before committing to a path. IME substitutes a numerical look-ahead: run the Value Strand on the assembled slate and combine its next-token log-likelihood with the controller prior over fragments to score each successor state. This reproduces ToT's "generate-evaluate-select" loop while staying in latent space, avoiding the prompt-length and variance costs of free-form textual judges. (Yao et al., 2023).

6.4 Budget knobs: mapping protocol hyperparameters to IME

Branching factor. ToT's "number of thoughts per step" ↔ IME's beam width B (number of assemblies retained). (Yao et al., 2023).

Depth. ToT's search depth ↔ number of IME steps before decode/emit. (Yao et al., 2023).

Voting/aggregation. Self-Consistency's majority vote ↔ beam aggregation (score marginalization or answer voting across beams). (Wang et al., 2022).

Decomposition hardness. LtM's step schedule ↔ candidate filtering and capacity C (how many fragments can co-reside in the slate during each phase). (Zhou et al., 2022).

6.5 Extending beyond text-only protocols: program- and tool-augmented thoughts

Methods like PAL replace parts of the reasoning chain with executable programs, letting the LLM plan in language but delegate computation to a runtime. In Okazaki-RAG, this can be internalized by treating program stubs and tool-call descriptors as fragment types; Assemble then builds mixed slates (facts + code), and look-ahead can execute to score states—bridging from CoT/ToT to "program-of-thought" within the same control loop. (Gao et al., 2022).

6.6 Why internalization helps

CoT/ToT deliver strong gains but incur token overhead, prompt sensitivity, and latency from generating and judging long textual traces. By moving the same operations to latent assemblies, IME: (i) reuses pre-computed fragments (no re-encode), (ii) searches under a hard working-memory capacity, and (iii) evaluates candidates with short look-ahead passes—preserving ToT-style exploration while reducing variability from verbose prompting. This design relies on the Okazaki-RAG substrate (Context Strand control + fragment assembly + late fusion). (Yao et al., 2023; Ghosh et al., 2025).

6.7 Auditing and explanations: traces without token chains

Although IME does not emit textual chains by default, its decision trace—the sequence (W_0, …, W_n), the ops taken, and scores—can be logged as a compact explanation, analogous to revealing a CoT/ToT tree after the fact. Each node carries fragment-level provenance (document IDs/offsets), enabling citation and replay—an advantage over free-form CoT text that may omit sources. (Ghosh et al., 2025).

Summary. CoT introduces sequential reasoning, Self-Consistency brings ensemble robustness, LtM imposes a decomposition schedule, and ToT adds branch-and-bound search. IME retains all four ideas but executes them inside the model by editing and scoring latent fragment assemblies under capacity constraints—turning external prompting protocols into a first-class architectural control regime.

7. Critical examination: latency/quality trade-offs, fidelity risks, provenance, and security

Okazaki-RAG deliberately moves work out of token space and into representation space. That shift buys interactive speed and tighter control—but only if we manage three failure surfaces: latency budgets, fidelity of mid-sequence KV injection, and trust (provenance & security). Below we dissect each, tie it to the serving primitives (EPIC/PIC, CacheBlend, TurboRAG), and spell out guardrails.

7.1 Latency vs. quality: where the time goes

- **Prefill avoidance is real headroom.** By precomputing per-chunk KV caches offline and retrieving KVs directly at inference, **TurboRAG** reports up to 9.4× lower TTFT (≈8.6× on average) versus text-first RAG. This headroom bankrolls IME's look-ahead scoring without a prompt-length tax. (Lu et al., 2024).
- **Assembly is cheap; repair costs are tunable. EPIC/PIC** make position-independent KV reuse possible (non-prefix), so IME can assemble candidate contexts without re-encoding. But quality repair (below) introduces small bursts of compute—the main knob is how much to recompute per boundary. (Hu et al., 2024).
- **Search is the new throttle.** IME's beam × depth × (repairs per candidate) determines end-to-end latency. Compared with prompt-level CoT/ToT (which spends tokens), Okazaki-RAG spends short forward passes on assembled KVs; the win shows up strongest when it would otherwise prefill long retrieved text. In workloads with numerous candidates or heavy repair, the gains can narrow. A practical recipe is: small beam (2–4), shallow depth (1–3), and boundary-only repairs. (Yao et al., 2023).
- **Reality check.** Long inputs still carry position bias—models under-use middle context—so flat prompt concatenation is not a free alternative to IME search. Selecting/assembling a small slate remains advantageous even for "long-context" LLMs. (Liu et al., 2023).

7.2 Fidelity risks of mid-sequence KV injection (and how to contain them)

Failure mode

If link independently computed caches, tokens in chunk B never attended chunk A when their KVs were first computed; naïve concatenation loses cross-attention and can drift semantics.

Mitigations

- **Position-independent linking (EPIC/PIC).** EPIC supplies the linker (e.g., AttnLink/LegoLink) to place KVs at arbitrary positions and partially re-establish attention structure. This makes discontinuous assembly feasible rather than merely prefix-safe caching. (Hu et al., 2024).
- **Selective recomputation (CacheBlend).** Instead of full re-encode, CacheBlend reuses caches and recomputes a small, chosen subset of tokens (often boundary/high-discrepancy) to repair attention—recovering quality with a small latency bump that can be pipelined with KV retrieval. (Yao et al., 2024).
- **KV-linking improvements.** New variants (e.g., KVLink) continue to report large TTFT cuts while improving accuracy, suggesting the repair frontier is active and improving. (Designs vary; use with measurement.)

Operational guidance

Prefer final-layer fragments (as in Okazaki-RAG) to minimize mismatch; repair only at assembly boundaries unless value gradients suggest deeper recompute; and cap slate size (C) to keep the number of boundaries—and thus repairs—small. EPIC's diagnostics and CacheBlend's discrepancy heuristics help instrument these choices. (Ghosh et al., 2025; Hu et al., 2024; Yao et al., 2024).

7.3 Provenance and attribution: preventing "source amnesia"

Representation-level composition can blur which fragment sourced which claim. That's a reputational and safety problem, especially in high-stakes domains.

What the literature says

- **RAG evaluation** work now treats citation/attribution as a first-class metric alongside relevance and faithfulness; surveys catalog methods and benchmarks. (Gao et al., 2023; Huang & Huang, 2024).
- **Answer attribution** methods (entailment/attention-trace/internals) and dedicated frameworks (e.g., RAGE, RAGAs) evaluate whether outputs correctly reference supporting passages.
- **Community tracks** (e.g., TREC RAG) emphasize nugget recall and citation coverage, pushing toward standardized, verifiable grounding.

Design implications for Okazaki-RAG.

- **Fragment-level telemetry.** Every fragment carries document id, offsets, license; assemblies propagate a provenance map so each span of generated text can be tied back to specific fragments (and, transitively, to source docs).
- **Exposure as UX.** Surface which fragments were in the final slate and why (controller scores + look-ahead values). This is the latent analogue of showing a CoT tree plus citations.
- **Evaluation discipline.** Adopt attribution metrics (coverage, precision) in addition to task accuracy; resist over-indexing on BLEU/ROUGE alone.

7.4 Security: multi-tenant cache risks and classic LLM threats

Two distinct vectors matter here.

(A) Multi-tenant KV sharing. Recent work (NDSS 2025) shows prompt leakage via KV-cache sharing across tenants (PROMPTPEEK) (Hou et al. 2025), reconstructing sensitive user prompts from shared caches. If your serving stack shares KVs across users to save memory/compute (as in some continuous batching systems), treat that as an attack surface, not an optimization. Hard isolate or cryptographically partition caches.

(B) Prompt injection (including indirect). OWASP's LLM Top-10 lists prompt injection as risk #1; in RAG/graph-RAG, indirect injection via retrieved or crawled content can subvert the controller and IME's selection policy (e.g., "ignore prior instructions; load this fragment"). Enforce allow-lists, content sanitization, and instruction-hierarchy checks at retrieval/assembly time.

Security posture for Okazaki-RAG.

- **Namespace isolation.** No cross-tenant KV sharing; assign tenant-scoped cache namespaces and authenticate fragment reads/writes. (Multi-tenant reuse should happen at text or embedding level, not at raw KV level.)
- **Signed fragments.** Persist content hashes + signatures with fragment metadata; verify on load; reject mutated or untrusted caches.

- **Policy checks in IME.** Before Append/Assemble, run lightweight content policy and URL/domain allow-list checks on candidate fragments; demote or drop items with adversarial markers.
- **Explainability as defense.** Log the state trace ($W_0 \to \cdots \to W_n$) with controller logits and look-ahead values; alarms on anomalous jumps help catch compromised selection policies.
- **Classic LLM hardening.** Apply OWASP guidance (least-privilege tools, output filters, egress control) to any program-of-thought or tool-fragments IME may execute.

7.5 What to measure (and how to fail safely)

- **Latency–quality frontier.** Report accuracy vs (beam, depth, repair-rate) under TurboRAG/EPIC/CacheBlend settings, plus TTFT/throughput. Don't claim "free" System-2; show the budget curve. (Lu et al., 2024; Hu et al., 2024; Yao et al., 2024).
- **Fidelity deltas.** Compare full prefill vs PIC + repair on the same assemblies; track error concentration at assembly boundaries. EPIC/CacheBlend both provide heuristics for selecting repair tokens—instrument and publish them. (Hu et al., 2024; Yao et al., 2024).
- **Attribution quality.** Evaluate citation coverage/precision alongside task metrics; where possible, align with community protocols (TREC RAG, RAGE/RAGAs).
- **Security drills.** Pen-test with indirect prompt injection corpora; simulate PROMPTPEEK-style cache attacks in a staging environment; demonstrate that namespace isolation prevents cross-tenant retrieval of KVs.

Bottom line. Okazaki-RAG's promise—deliberation at representation level—rests on measured use of EPIC/PIC (to assemble), CacheBlend (to repair), and TurboRAG (to precompute). If you (i) budget search/repair, (ii) prove fidelity at boundaries, (iii) surface fragment-level provenance, and (iv) hard-isolate caches while defending against prompt injection, you get a controllable System-2 loop that is fast and trustworthy. Skimp on any one of these, and the benefits can evaporate into latency spikes, quiet accuracy drift, or avoidable risk.

8. Contributions and research trajectories

This work advances a representation-level substrate for deliberation. Concretely, we: (i) formalized a dual-strand architecture that separates a lightweight **Context Strand** (global control/state) from a high-fidelity **Value Strand** (local reasoning), (ii) defined **Okazaki fragments**—precomputed final-layer KV blocks with summary handles—as actionable units of knowledge, and (iii) introduced the **Inline Manipulation Engine (IME)** to operate over a capacity-bounded slate via {Append, Remove, Assemble, Swap} with beam-bounded search and look-ahead scoring. We further grounded feasibility in modern serving primitives: **position-independent context caching (EPIC/PIC)** for non-prefix assembly, **selective KV recompute (CacheBlend)** for quality repair, and **KV-first retrieval (TurboRAG)** that collapses prefill latency—together enabling fragment-level search inside decoding rather than via long textual prompts. (Hu et al., 2024; Yao et al., 2024; Lu et al., 2024).

8.1 What this paper contributes

- **A cognitively constrained control regime.** We hard-bound the working set $|W_t| \leq C$ to emulate human working-memory limits (Miller's 7±2; Cowan's ≈4)—a design pressure that forces selectivity and enables tractable search. (Miller, 1956; Cowan, 2001).

- **Latent CoT/ToT internalization.** IME reinterprets Chain-of-Thought and Tree-of-Thoughts as latent edits to assembled fragments rather than long token traces, keeping the benefits of branching and self-evaluation while avoiding prompt bloat. (Wei et al., 2022; Yao et al., 2023).
- **Serving-aligned execution.** The Assemble primitive rides PIC/EPIC for non-prefix KV reuse, CacheBlend for boundary repair, and TurboRAG for offline text→KV conversion and large TTFT cuts—practical enablers for inline search. (Hu et al., 2024; Yao et al., 2024; Lu et al., 2024).
- **A principled alternative to "flat" long contexts.** Rather than relying on concatenation that suffers lost-in-the-middle effects, we compose a small, scored slate that the Value Strand can exploit. (Liu et al., 2023).

8.2 Training the controller: from heuristics to policies

- The Context Strand is a policy over fragments; turning it from a heuristic scorer into a trained controller is the central next step.
- **Imitation from search traces.** Run oracle searches (wide beam/depth with CacheBlend repairs), log ($W_{t,op}$) sequences and scores, and behavior-clone the controller to predict next moves/allocations. Traces can be seeded with ToT-style exploration and Self-Consistency ensembles to diversify positives. (Yao et al., 2023; Wang et al., 2022).
- **Process-supervised RL.** Replace outcome-only rewards with process rewards that grade intermediate assemblies (e.g., boundary fidelity, local likelihood, verifier checks). Prior work shows process reward models (PRMs) and process supervision outperform outcome-only signals on reasoning; we propose using PRMs as critics for IME states. (Lightman et al., 2023).
- **Generative/verification-aided PRMs.** Recent PRM variants combine CoT with code/verification and scale to large supervision corpora; these are natural drop-in critics for fragment assembly quality, with latency-aware penalties. (Zhao et al., 2025).
- **Verbal/self-improvement loops.** Methods like Reflexion show gains from reflective self-feedback. In our setting, reflections can update controller priors (over summaries) rather than just text, tightening proposal distributions for the next step. (Shinn et al., 2023).
- **Objective shaping.** Optimize a utility–cost objective that trades task score against deliberation budget (beam, depth, repair-rate). This aligns with adaptive computation principles: spend more compute on hard instances, less on easy ones.

8.3 Adaptive deliberation: budgeting compute at inference time

Beyond how to choose fragments is *how much* search to run per instance.

- **Difficulty-aware halting.** Borrowing from Adaptive Computation Time (ACT) and modern adaptive test-time compute, learn halting/expansion policies that predict whether more beams or deeper assemblies will pay off—potentially mid-generation.
- **Uncertainty- and consensus-driven scaling.** Use dispersion across beams (analogue of Self-Consistency variance) to trigger selective beam widening or additional repair at boundaries; collapse when consensus is high. (Wang et al., 2022).
- **Budget-constrained planning.** Formulate deliberation as a knapsack over compute primitives (assemble, repair, look-ahead): policy decides next marginal unit of compute under a latency cap—an emerging theme in efficient TTC surveys.

- **Program-of-Thought fragments.** Extend the library to include PAL/PoT fragments (code/tool stubs). Adaptive policies then decide when to execute tools vs. continue latent assembly, balancing verification gains against latency. (Gao et al., 2022).

8.4 Evaluation protocols and milestones

- **Compute–quality frontiers.** Report accuracy vs. TTFT/throughput while sweeping (beam, depth, repair-rate) under EPIC/CacheBlend/TurboRAG settings; show that controller learning shifts the Pareto frontier outward. (Lu et al., 2024; Hu et al., 2024; Yao et al., 2024).
- **Boundary fidelity audits.** Compare full prefill against PIC + repair on the same assemblies to quantify residual error at ligation sites (where quality risk concentrates).
- **Long-context stress.** Measure robustness to positional shifts (begin/middle/end) to verify that capacity-bounded assembly mitigates lost-in-the-middle failures. (Liu et al., 2023).

8.5 Outlook

Architecturally, Okazaki-RAG turns knowledge into compute blocks and deliberation into latent search—then anchors both in serving tech that already exists and is improving fast (PIC/EPIC, CacheBlend, TurboRAG). Methodologically, the road ahead is to train the controller as a policy (imitation → PRM-guided RL) and to adaptively budget deliberation at inference time (ACT-style halting, uncertainty-driven beam control). If these two lines converge, we expect System-2 behavior at lower, instance-adaptive cost—a more stable alternative to ever-longer prompts or uniformly heavy test-time computation.

The contemporary landscape of large language model (LLM) architectures is defined by a fundamental tension between two dominant paradigms for incorporating external knowledge: the expansion of model context windows to unprecedented lengths and the refinement of Retrieval-Augmented Generation (RAG) systems. This tension is not merely a technical trade-off but a systemic conflict involving computational economics, the preservation of semantic coherence, and the fidelity of factual grounding. The Context-Value-Value Context-RAG (CVVC-RAG) architecture emerges from this conflict, proposing not a simple compromise but a new architectural paradigm rooted in the principle of decoupling, which redefines the roles and responsibilities within the generation process. A companion paper to this is under preparation.

References

[1] Ahmad, Sarat, et al. "Benchmarking Vector, Graph and Hybrid Retrieval Augmented Generation (RAG) Pipelines for Open Radio Access Networks (ORAN)." ArXiv, 2025, https://arxiv.org/abs/2507.03608.

[2] Arslan, Muhammad, et al. "A Survey on RAG with LLMs." Procedia Computer Science, vol. 246, 2023, pp. 3781-3790, https://doi.org/10.1016/j.procs.2024.09.178.

[3] Balakrishnan, Lata, and Robert A. Bambara. "Okazaki Fragment Metabolism." Cold Spring Harbor Perspectives in Biology, vol. 5, no. 2, 2013, p. a010173, https://doi.org/10.1101/cshperspect.a010173.

[4] Blair, Kerry, et al. "Mechanism of Human Lig1 Regulation by PCNA in Okazaki Fragment Sealing." Nature Communications, vol. 13, no. 1, 2022, pp. 1-15, https://doi.org/10.1038/s41467-022-35475-z.

[5] Borgeaud, Sebastian, et al. "Improving Language Models by Retrieving from Trillions of Tokens." ArXiv, 2021, https://arxiv.org/abs/2112.04426.

[6] Bornschein, Jorg, and Yazhe Li. "Transformers for Supervised Online Continual Learning." ArXiv, 2024, https://arxiv.org/abs/2403.01554.

[7] Burgers, Peter M. "Solution to the 50-year-old Okazaki-fragment Problem." Proceedings of the National Academy of Sciences, vol. 116, no. 9, 2019, pp. 3358-3360, https://doi.org/10.1073/pnas.1900372116.
[8] Chan, Brian J., et al. "Don't Do RAG: When Cache-Augmented Generation Is All You Need for Knowledge Tasks." ArXiv, 2024, https://doi.org/10.1145/3701716.3715490.
[9] Canal, Berta, et al. "The DNA Replication Checkpoint Limits Okazaki Fragment Accumulation to Protect and Restart Stalled Forks." Molecular Cell, vol. 85, no. 13, 2025, pp. 2462-2473.e6, https://doi.org/10.1016/j.molcel.2025.06.001.
[10] Chen, Xinquan, et al. "VRPRM: Process Reward Modeling via Visual Reasoning." ArXiv, 2025, https://arxiv.org/abs/2508.03556.
[11] Cheng, Xiaoxue, et al. "Incentivizing Dual Process Thinking for Efficient Large Language Model Reasoning." ArXiv, 2025, https://arxiv.org/abs/2505.16315.
[12] Chu, Zheng, et al. "Navigate through Enigmatic Labyrinth A Survey of Chain of Thought Reasoning: Advances, Frontiers and Future." ArXiv, 2023, https://arxiv.org/abs/2309.15402.
[13] Cowan, Nelson. "George Miller'S Magical Number of Immediate Memory in Retrospect: Observations on the Faltering Progression of Science." Psychological Review, vol. 122, no. 3, 2015, p. 536, https://doi.org/10.1037/a0039035.
[14] Cowan N. (2001) The magical number 4 in short-term memory: a reconsideration of mental storage capacity. Behav Brain Sci. 2001 Feb;24(1):87-114; discussion 114-85. PMID: 11515286. https://doi.org/10.1017/s0140525x01003922
[15] Dai, Zihang, et al. "Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context." ArXiv, 2019, https://arxiv.org/abs/1901.02860.
[16] Difilippantonio, Michael J., et al. "RAG1 Mediates Signal Sequence Recognition and Recruitment of RAG2 in V(D)J Recombination." Cell, vol. 87, no. 2, 1996, pp. 253-262, https://doi.org/10.1016/S0092-8674(00)81343-4.
[17] Dovrat, Daniel, et al. "Sequential Switching of Binding Partners on PCNA during in Vitro Okazaki Fragment Maturation." Proceedings of the National Academy of Sciences, vol. 111, no. 39, 2014, pp. 14118-14123, https://doi.org/10.1073/pnas.1321349111.
[18] Fayyazi, Reza, et al. "ProveRAG: Provenance-Driven Vulnerability Analysis with Automated Retrieval-Augmented LLMs." ArXiv, 2024, https://arxiv.org/abs/2410.17406.
[19] Ficek, Aleksander, et al. "GPT Vs RETRO: Exploring the Intersection of Retrieval and Parameter-Efficient Fine-Tuning." ArXiv, 2024, https://arxiv.org/abs/2407.04528.
[20] Fu, Jinlan, et al. "Hint-before-Solving Prompting: Guiding LLMs to Effectively Utilize Encoded Knowledge." ArXiv, 2024, https://arxiv.org/abs/2402.14310.
[21] Gao, Yunfan, et al. "Retrieval-Augmented Generation for Large Language Models: A Survey." ArXiv, 2023, https://arxiv.org/abs/2312.10997.
[22] Gao, Luyu, et al. "PAL: Program-aided Language Models." ArXiv, 2022, https://arxiv.org/abs/2211.10435.
[23] Gao, Yunfan, et al. "Modular RAG: Transforming RAG Systems into LEGO-like Reconfigurable Frameworks." ArXiv, 2024, https://arxiv.org/abs/2407.21059.
[24] Ghosh D, Ghosh D, Ghosh D (2025) Okazaki-RAG: A Dual-Strand, Parameter-Efficient Architecture for Long-Context Modeling, https://www.researchgate.net/publication/394407351_Okazaki-RAG_A_Dual-Strand_Parameter-Efficient_Architecture_for_Long-Context_Modeling, http://dx.doi.org/10.13140/RG.2.2.12473.89444
[25] Gronchi, Giorgio, and Axel Perini. "Dual-process Theories of Thought As Potential Architectures for Developing Neuro-symbolic AI Models." Frontiers in Cognition, vol. 3, 2024, p. 1356941, https://doi.org/10.3389/fcogn.2024.1356941.
[26] Gupta, Shailja, et al. "A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions." ArXiv, 2024, https://arxiv.org/abs/2410.12837.
[27] He, Junqing, et al. "Never Lost in the Middle: Mastering Long-Context Question Answering with Position-Agnostic Decompositional Training." ArXiv, 2023, https://arxiv.org/abs/2311.09198.
[28] Hiraizumi, Masahiro, et al. "Structural Mechanism of Bridge RNA-guided Recombination." Nature, vol. 630, no. 8018, 2024, pp. 994-1002, https://doi.org/10.1038/s41586-024-07570-2.
[29] Hou, Xinyi, et al. "Unveiling the Landscape of LLM Deployment in the Wild: An Empirical Study." ArXiv, 2025, https://arxiv.org/abs/2505.02502.

[30] Hsieh, Cheng, et al. "Found in the Middle: Calibrating Positional Attention Bias Improves Long Context Utilization." ArXiv, 2024, https://arxiv.org/abs/2406.16008.
[31] Hu, Junhao, et al. "EPIC: Efficient Position-Independent Context Caching for Serving Large Language Models." ArXiv, 2024, https://arxiv.org/abs/2410.15332.
[32] Huang, Yizheng, and Jimmy Huang. "A Survey on Retrieval-Augmented Text Generation for Large Language Models." ArXiv, 2024, https://arxiv.org/abs/2404.10981.
[33] Hui, Wenyang, et al. "RoT: Enhancing Large Language Models with Reflection on Search Trees." ArXiv, 2024, https://arxiv.org/abs/2404.05449.
[34] Kahneman, D. (2011). *Thinking, fast and slow*. Farrar, Straus and Giroux. https://psycnet.apa.org/record/2011-26535-000
[35] Klett, Phoebe, and Thomas Ahle. "Extended Mind Transformers." ArXiv, 2024, https://arxiv.org/abs/2406.02332.
[36] Kojima, Takeshi, et al. "Large Language Models Are Zero-Shot Reasoners." ArXiv, 2022, https://arxiv.org/abs/2205.11916.
[37] Lee, Hyungwoo, et al. "Shared Disk KV Cache Management for Efficient Multi-Instance Inference in RAG-Powered LLMs." ArXiv, 2025, https://arxiv.org/abs/2504.11765.
[38] Leng, Quinn, et al. "Long Context RAG Performance of Large Language Models." ArXiv, 2024, https://arxiv.org/abs/2411.03538.
[39] Lewis, Patrick, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." ArXiv, 2020, https://arxiv.org/abs/2005.11401.
[40] Lightman, Hunter, et al. "Let's Verify Step by Step." ArXiv, 2023, https://arxiv.org/abs/2305.20050.
[41] Liu, Nelson F., et al. "Lost in the Middle: How Language Models Use Long Contexts." ArXiv, 2023, https://arxiv.org/abs/2307.03172.
[42] Liu, Xiang, et al. "Can LLMs Maintain Fundamental Abilities under KV Cache Compression?" ArXiv, 2025, https://arxiv.org/abs/2502.01941.
[43] Liu, Jia, et al. "State Machine of Thoughts: Leveraging Past Reasoning Trajectories for Enhancing Problem Solving." ArXiv, 2023, https://arxiv.org/abs/2312.17445.
[44] Lowe, Scott C. "System 2 Reasoning Capabilities Are Nigh." ArXiv, 2024, https://arxiv.org/abs/2410.03662.
[45] Lu, Songshuo, et al. "TurboRAG: Accelerating Retrieval-Augmented Generation with Precomputed KV Caches for Chunked Text." ArXiv, 2024, https://arxiv.org/abs/2410.07590.
[46] Maheshwari, Harsh, et al. "CiteFix: Enhancing RAG Accuracy Through Post-Processing Citation Correction." ArXiv, 2025, https://arxiv.org/abs/2504.15629.
[47] Matsumoto, Yoshihiro, et al. "Dynamic DNA-bound PCNA Complexes Co-ordinate Okazaki Fragment Synthesis, Processing and Ligation." Journal of Molecular Biology, vol. 432, no. 24, 2020, p. 166698, https://doi.org/10.1016/j.jmb.2020.10.032.
[48] Miller, G. A. (1956). The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review, 63*(2), 81–97. https://doi.org/10.1037/0033-295x.101.2.343
[49] Munkhdalai, Tsendsuren, et al. "Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention." ArXiv, 2024, https://arxiv.org/abs/2404.07143.
[50] Ni, Bo, et al. "Towards Trustworthy Retrieval Augmented Generation for Large Language Models: A Survey." ArXiv, 2025, https://arxiv.org/abs/2502.06872.
[51] Norlund, Tobias, et al. "On the Generalization Ability of Retrieval-Enhanced Transformers." ArXiv, 2023, https://arxiv.org/abs/2302.12128.
[52] Oche, Agada J., et al. "A Systematic Review of Key Retrieval-Augmented Generation (RAG) Systems: Progress, Gaps, and Future Directions." ArXiv, 2025, https://arxiv.org/abs/2507.18910.
[53] Pleines, Marco, et al. "Memory Gym: Towards Endless Tasks to Benchmark Memory Capabilities of Agents." ArXiv, 2023, https://arxiv.org/abs/2309.17207.
[54] Raducanu, Stefan, et al. "Mechanistic Investigation of Human Maturation of Okazaki Fragments Reveals Slow Kinetics." Nature Communications, vol. 13, 2022, p. 6973, https://doi.org/10.1038/s41467-022-34751-2.
[55] Rashiti, Gentiana, et al. "Retro-li: Small-Scale Retrieval Augmented Generation Supporting Noisy Similarity Searches and Domain Shift Generalization." ArXiv, 2024, https://doi.org/10.3233/FAIA240837.

[56] Rensink Ronald A. (2001). Four-sight in hindsight: The existence of magical numbers in vision, Behavioral and Brain Sciences, Volume 24, Issue 1, February 2001, pp. 141 – 142, https://doi.org/10.1017/S0140525X01483920.
[57] Rubin, Ohad, and Jonathan Berant. "Retrieval-Pretrained Transformer: Long-range Language Modeling with Self-retrieval." ArXiv, 2023, https://arxiv.org/abs/2306.13421.
[58] Salemi, Alireza, and Hamed Zamani. "Comparing Retrieval-Augmentation and Parameter-Efficient Fine-Tuning for Privacy-Preserving Personalization of Large Language Models." ArXiv, 2024, https://arxiv.org/abs/2409.09510.
[59] She, Shuaijie, et al. "R-PRM: Reasoning-Driven Process Reward Modeling." ArXiv, 2025, https://arxiv.org/abs/2503.21295.
[60] Shinn, Noah, et al. "Reflexion: Language Agents with Verbal Reinforcement Learning." ArXiv, 2023, https://arxiv.org/abs/2303.11366.
[61] Song, Linke, et al. "The Early Bird Catches the Leak: Unveiling Timing Side Channels in LLM Serving Systems." ArXiv, 2024, https://arxiv.org/abs/2409.20002.
[62] Su, Weihang, et al. "Parametric Retrieval Augmented Generation." ArXiv, 2025, https://arxiv.org/abs/2501.15915.
[63] Sun, Haitao, et al. "Okazaki Fragment Maturation: DNA Flap Dynamics for Cell Proliferation and Survival." Trends in Cell Biology, vol. 33, no. 3, 2022, p. 221, https://doi.org/10.1016/j.tcb.2022.06.014.
[64] Tian, Yuhui, et al. "Structural Insight into Okazaki Fragment Maturation Mediated by PCNA-bound FEN1 and RNaseH2." The EMBO Journal, vol. 44, no. 2, 2024, p. 484, https://doi.org/10.1038/s44318-024-00296-x.
[65] Tworkowski, Szymon, et al. "Focused Transformer: Contrastive Training for Context Scaling." ArXiv, 2023, https://arxiv.org/abs/2307.03170.
[66] Uhlmann, Frank. "A Unified Model for Cohesin Function in Sister Chromatid Cohesion and Chromatin Loop Formation." Molecular Cell, vol. 85, no. 6, 2025, pp. 1058-1071, https://doi.org/10.1016/j.molcel.2025.02.005.
[67] Ulitsky, Igor, and David P. Bartel. "LincRNAs: Genomics, Evolution, and Mechanisms." Cell, vol. 154, no. 1, 2013, pp. 26-46, https://doi.org/10.1016/j.cell.2013.06.020.
[68] Wang, Boxin, et al. "InstructRetro: Instruction Tuning Post Retrieval-Augmented Pretraining." ArXiv, 2023, https://arxiv.org/abs/2310.07713.
[69] Wang, Xuezhi, et al. "Self-Consistency Improves Chain of Thought Reasoning in Language Models." ArXiv, 2022, https://arxiv.org/abs/2203.11171.
[70] Wang, Han, et al. "Soft Self-Consistency Improves Language Model Agents." ArXiv, 2024, https://arxiv.org/abs/2402.13212.
[71] Wei, Jason, et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." ArXiv, 2022, https://arxiv.org/abs/2201.11903.
[72] Wikipedia, (2025). The Magical Number Seven, Plus or Minus Two, https://en.wikipedia.org/wiki/The_Magical_Number_Seven,_Plus_or_Minus_Two
[73] Wikipedia, (2025). Dual process theory, https://en.wikipedia.org/wiki/Dual_process_theory
[74] Wikipedia, (2025). Thinking, Fast and Slow, https://en.wikipedia.org/wiki/Thinking%2C_Fast_and_Slow
[75] Williams, Jessica S., et al. "High-fidelity DNA Ligation Enforces Accurate Okazaki Fragment Maturation during DNA Replication." Nature Communications, vol. 12, no. 1, 2021, pp. 1-11, https://doi.org/10.1038/s41467-020-20800-1.
[76] Wu, Yuhuai, et al. "Memorizing Transformers." ArXiv, 2022, https://arxiv.org/abs/2203.08913.
[77] Xu, Silei, et al. "Chain of Draft: Thinking Faster by Writing Less." ArXiv, 2025, https://arxiv.org/abs/2502.18600.
[78] Yang, Huan, et al. "KVShare: An LLM Service System with Efficient and Effective Multi-Tenant KV Cache Reuse." ArXiv, 2025, https://arxiv.org/abs/2503.16525.
[79] Yao, Shunyu, et al. "Tree of Thoughts: Deliberate Problem Solving with Large Language Models." ArXiv, 2023, https://arxiv.org/abs/2305.10601.
[80] Yao, Jiayi, et al. "CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion." ArXiv, 2024, https://arxiv.org/abs/2405.16444.
[81] Yu, Hao, et al. "Evaluation of Retrieval-Augmented Generation: A Survey." ArXiv, 2024, https://doi.org/10.1007/978-981-96-1024-2_8.

[82] Yu, Yijiong, et al. "Mitigate Position Bias in Large Language Models via Scaling a Single Dimension." ArXiv, 2024, https://arxiv.org/abs/2406.02536.
[83] Zamarreño, Javier, et al. "Timely Lagging Strand Maturation Relies on Ubp10 Deubiquitylase-mediated PCNA Dissociation from Replicating Chromatin." Nature Communications, vol. 15, no. 1, 2024, pp. 1-18, https://doi.org/10.1038/s41467-024-52542-9.
[84] Zhang, Yufeng, et al. "Pattern-Aware Chain-of-Thought Prompting in Large Language Models." ArXiv, 2024, https://arxiv.org/abs/2404.14812.
[85] Zhang, Zhaoyang, et al. "Cached Transformers: Improving Transformers with Differentiable Memory Cache." ArXiv, 2023, https://arxiv.org/abs/2312.12742.
[86] Zhang, Zhenru, et al. "The Lessons of Developing Process Reward Models in Mathematical Reasoning." ArXiv, 2025, https://arxiv.org/abs/2501.07301.
[87] Zhang, Shao, et al. "Leveraging Dual Process Theory in Language Agent Framework for Real-time Simultaneous Human-AI Collaboration." ArXiv, 2025, https://arxiv.org/abs/2502.11882.
[88] Zhao, Jian, et al. "GenPRM: Scaling Test-Time Compute of Process Reward Models via Generative Reasoning." ArXiv, 2025, https://arxiv.org/abs/2504.00891.
[89] Zhao, Qingfei, et al. "Towards Multi-Source Retrieval-Augmented Generation via Synergizing Reasoning and Preference-Driven Retrieval." ArXiv, 2024, https://arxiv.org/abs/2411.00689.
[90] Zhao, Qingfei, et al. "LongRAG: A Dual-Perspective Retrieval-Augmented Generation Paradigm for Long-Context Question Answering." ArXiv, 2024, https://arxiv.org/abs/2410.18050.
[91] Zhao, Penghao, et al. "Retrieval-Augmented Generation for AI-Generated Content: A Survey." ArXiv, 2024, https://arxiv.org/abs/2402.19473.
[92] Zhao, Shiju, et al. "MPIC: Position-Independent Multimodal Context Caching System for Efficient MLLM Serving." ArXiv, 2025, https://arxiv.org/abs/2502.01960.
[93] Zhou, Denny, et al. "Least-to-Most Prompting Enables Complex Reasoning in Large Language Models." ArXiv, 2022, https://arxiv.org/abs/2205.10625.
[94] Zhou, Jinan, et al. "BAR Conjecture: The Feasibility of Inference Budget-Constrained LLM Services with Authenticity and Reasoning." ArXiv, 2025, https://arxiv.org/abs/2507.23170.
[95] Zhou, Pei, et al. "Self-Discover: Large Language Models Self-Compose Reasoning Structures." ArXiv, 2024, https://arxiv.org/abs/2402.03620.
[96] Ziabari, Alireza S., et al. "Reasoning on a Spectrum: Aligning LLMs to System 1 and System 2 Thinking." ArXiv, 2025, https://arxiv.org/abs/2502.12470.